Deep learning long-range information in undirected graphs with wave networks

Matthew K. Matlock*, Arghya Datta*, Na Le Dang*, Kevin Jiang*, S. Joshua Swamidass*†

*Department of Pathology and Immunology
Washington University in Saint Louis
Saint Louis, Missouri, USA

†Email: swamidass@wustl.edu

***Abstract*—Graph algorithms are key tools in many fields of science and technology. Some of these algorithms depend on propagating information between distant nodes in a graph. Recently, there have been a number of deep learning architectures proposed to learn on undirected graphs. However, most of these architectures aggregate information in the local neighborhood of a node, and therefore they may not be capable of efficiently propagating long-range information. To solve this problem we examine a recently proposed architecture, wave, which propagates information back and forth across an undirected graph in waves of nonlinear computation. We compare wave to graph convolution, an architecture based on local aggregation, and find that wave learns three different graph-based tasks with greater efficiency and accuracy. These three tasks include (1) labeling a path connecting two nodes in a graph, (2) solving a maze presented as an image, and (3) computing voltages in a circuit. These tasks range from trivial to very difficult, but wave can extrapolate from small training examples to much larger testing examples. These results show that wave may be able to efficiently solve a wide range of tasks that require long-range information propagation across undirected graphs. An implementation of the wave network, and example code for the maze task are included in the tflon deep learning toolkit (https://bitbucket.org/mkmatlock/tflon).**

I. Introduction

Deep learning is a powerful approach to machine learning in domains with complex data types [1]–[4]. Modeling relationships among objects is a key analytic task in many fields. These relationships give rise to a natural graph data structure, where objects are nodes and relationships are edges, each of which can have properties. Recently, deep learning architectures, usually variants of convolutional neural networks, have been adapted for graph data [5], [6]. Convolutional neural networks have become a standard tool for machine learning in euclidean domains, such as images [7], [8], videos [9], and 3-d volumetric data [10], [11]. These approaches rely upon the key assumption that nodes can be efficiently described by local aggregation, either by summarizing the local graph topology, or by message passing between adjacent nodes. A recent review of these approaches acknowledges that long-range information is tacitly ignored [12]. Graph convolution only aggregates information locally; it does not efficiently propagate long-range information across a graph.

Algorithms operating on undirected graphs are key tools in nearly every field of science and technology. Notable examples include internet search [13], bibliometrics [14], [15], social sciences [16], cell biology [17], [18], genomics [19], chemistry [20], [21], circuit design [22], and transportation engineering [23]. Some of these algorithms produce output for every edge or node, and small changes to the graph can have far-reaching impact on the output. For example, long-range information is required to determine whether nodes are in a cycle. Efficient algorithms for ring detection are well known [20]; local information alone, however, is not sufficient to solve this task. For many algorithms commonly used on graph data, it is difficult to intuit a solution based only on local aggregation, because information must be propagated across the whole graph.

In this study, we compare wave, an architecture designed to efficiently propagate information across a graph, with graph convolution. Wave has recently been shown to outperform graph convolution in modeling aromatic and conjugate system size, two fundamentally important graph algorithms in chemical informatics [24]. Expanding upon that work, we show that wave outperforms graph convolution on three tasks: labeling a path connecting two nodes in a graph, (2) solving a maze presented as an image, and (3) computing voltages in a circuit. We chose these tasks because they exhibit important, long-range dependencies between two or more nodes in the graph, and because these problems have existing algorithmic solutions. If deep learning architectures are to successfully understand graph structured data, we would at minimum expect them to be able to solve these and other algorithmic tasks. These three tasks range in difficulty, from trivial to very difficult. Wave achieves better accuracy and efficiency than graph convolution on all of these tasks, suggesting that wave might have value in a large range of tasks where undirected graphs are used to represent data.

II. Related work

Techniques for representing graph structured data for input to deep learning architectures can be broadly separated into three classes: hand engineered features, unsupervised learning,

978-1-7281-2009-6/$31.00 ©2019 IEEE paper N-20288.pdf

and supervised learning. These approaches almost exclusively focus on finding node features that characterize local topology.

Historically, hand engineered features were commonly used because techniques for automated representation learning did not exist. Hand engineered features describe the local topology near a node in terms of its degree, and the connectivity properties of subgraphs formed by extracting a node, its neighbors, and their collected edges [25]–[28].

Several unsupervised techniques for graphs utilize random walks on local neighborhoods of a node to generate data for learning fixed-length vector embeddings. DeepWalk [29] adapts the word2vec [30] approach for word embeddings by modeling nodes as words and random walks as sentences. LINE [31], extends the random walk approach by introducing an objective that preserves properties of the local topology. Node2vec [32] adds a more flexible definition of node neighborhoods realized by random walks. These unsupervised embedding approaches are closely related to graph kernels, which define a similarity metric between nodes that can be used to impute labels on similar nodes [33], [34].

Supervised approaches utilize custom neural network components that can operate directly on graphs. Traditional convolutional neural networks require uniform spatial or temporal sampling of data, while graphs have flexible spatial structure. Several works generate receptive fields for traditional neurons by aggregating node and edge features over local neighborhoods [5], [35]. Extending upon these approaches, specialized graph convolution architectures have also been proposed [6], [36], [37]. These architectures use learnable message functions, which allow nodes to communicate with one another along their shared edges [38].

Two key works have introduced recurrent neural networks operating on graphs. Tree LSTMs operate on trees (directed graphs without cycles), aggregating data from multiple child nodes to a parent node. Tree LSTMs achieve state of the art performance on language modeling tasks [39]. Inner recursive neural networks operate on undirected graphs by enumerating a unique spanning tree for each node and applying an architecture similar to Tree LSTMs. These networks have shown promise in several chemical informatics tasks [40]. Unfortunately, neither of these techniques can operate on undirected graphs with cycles without loss of information.

In addition to graph specific architectures, a general purpose architecture, the differentiable neural computer [41], [42], can enumerate paths connecting nodes in a graph. Unfortunately, this architecture must be trained by reinforcement learning, a huge computational cost, and it does not scale efficiently to very large graphs.

Finally, while we deal only with static graphs, recent works have addressed the problem of computing on dynamic graphs, which change over time [43]. A variant of this problem, in which graph structures evolve during computation, has been identified as an important outstanding problem in deep learning [44], [45]. A deep learning algorithm capable of inferring relationships and generating dynamic graph structures may substantially expand the applications of deep learning into general reasoning domains. These works do not specifically address the problem of long-range information propagation in graphs.

III. Methods

A. Graph convolution networks

Graph convolution has shown promising results in subgraph isomorphism [46], program verification [36], chemical informatics [5], [6], and quantum chemistry [38]. Graph convolution learns node representations by repeated aggregation over local neighborhoods (Figure 1). The state update on pass p for a node u with neighbors $v \in N$ is defined as:

$$S_u^p = R_1^p(S_u^{p-1}, M_u^p), \tag{1}$$

where R_1^p is a recurrent neural network, which may be parameterized differently for each pass, S_u^{p-1} is the node state from the prior pass, and M_u^p is a message function, defined by,

$$M_u^p = \sum_{v \in N} c_1^p(E_{uv}^{p-1}, S_v^{p-1}), \tag{2}$$

where c_1^p is a neural network, and E_{uv}^{p-1} is the state of edge u, v from the prior pass. The edge states are updated by the recurrence

$$E_{uv}^p = R_2^p(E_{uv}^{p-1}, M_{uv}^p). \tag{3}$$

Once again, M_{uv}^p is the output of a message function, defined by

$$M_{uv}^p = c_2^p(S_u^{p-1}, S_v^{p-1}) + c_2^p(S_v^{p-1}, S_u^{p-1}), \tag{4}$$

where c_2^p is another neural network. In this work, the R_*^p, and c_*^p are each dense layers with exponential linear (ELU) activation [47]. Additional details can be found in Kearnes *et al* [6]. Graph convolution requires as many passes as the radius of a graph to propagate information between all nodes [24].

B. Wave networks

In wave, similar to graph convolution, node states are updated based on the prior node states and a message constructed from neighbor states (Equation 1). However, updates are ordered by breadth first search, and information is propagated from a central node in a wave across the entire graph (Figure 1). Updates are executed in forward and backward passes. During a forward pass, the node update depends only upon ancestor nodes, and the order is reversed in the backward pass. This node ordering enables information to traverse the entire graph in a single pass.

The wave message function is computed by a special mix network, which only uses information from incoming nodes I. Formally, given a node u with incoming nodes $v \in I$, the message function M_u^p for pass p is,

$$M_u^p = b + \sum_{v \in I} w \odot S_u^{p-1} \odot \frac{n_{1,\pi(u,v)}(S_u^{p-1}, E_{uv}, S_v^p)}{\sum_{v \in I} n_{1,\pi(u,v)}(S_u^{p-1}, E_{uv}, S_v^p)} + \sum_{v \in I} S_u^{p-1} \odot n_{2,\pi(u,v)}(S_u^{p-1}, E_{uv}, S_v^p), \tag{5}$$

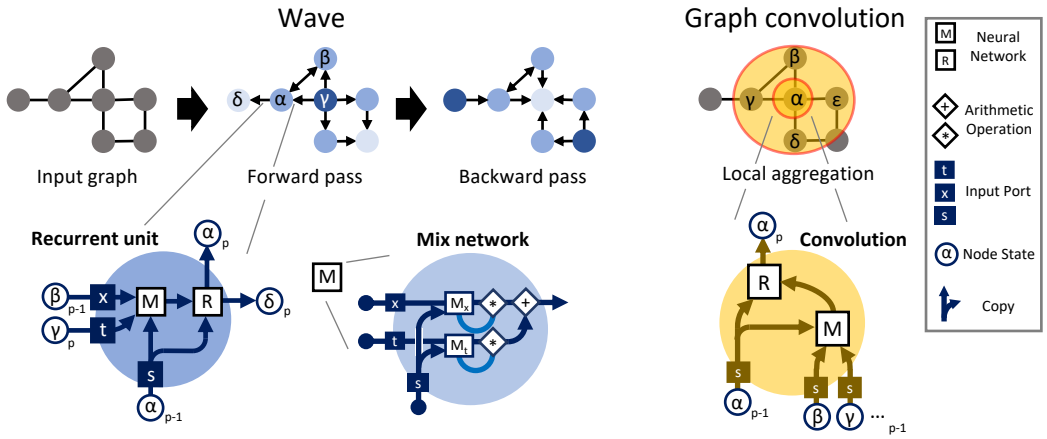

Fig. 1. **Wave enables long-range information propagation in graph structured data.** Two alternative paradigms for computing node representations in graphs: wave (left, blue) and graph convolution (right, orange). Graph convolution is a member of a general class of methods based on local aggregation of information. For each convolutional pass, updating a node depends on the node itself and its neighbors. Sufficient passes must be computed to ensure all nodes share information. In contrast, wave passes information forward and backward across a graph in a single pass. To achieve this, nodes and edges are ordered by breadth first search, starting from a central node. Each node update depends upon its immediate ancestors, and is propagated to descendant nodes.

where $\odot$ is element-wise vector multiplication, $\pi(u, v)$ is an index (0 or 1) chosen by the type of edge in the breadth first search (tree or cross), the $n_{1,\pi(u,v)}$ are neural networks with exponential activation, the $n_{2,\pi(u,v)}$ are neural networks with softsign activation [48], w is a weight vector, and b is a bias vector. For cross edges, the sibling node state from the previous pass is used, as the current pass state is not yet available. Informally, the mix network computes two weighted sums of the input states: one weighted by a neural network with softmax output and one weighted by a neural network with softsign output. The mix network can range between computing a weighted sum, a maximum, or a minimum of the input states, depending upon its parameters. Additional details of this architecture can be found in Matlock *et al* [24].

C. miniGRU

In Matlock *et al* [24], the default recurrent unit (R^p) for wave was a variant of the gated recurrent unit (GRU) [49], [50], which we refer to as miniGRU. Briefly, a GRU is a neural memory cell consisting of three gates: read, update, and output. The miniGRU differs by omitting the read gate, which is redundant with the mix network. Formally, given a input x_{p-1} and memory state s_{p-1}, the recurrence equations for pass p are:

$$\begin{aligned} u_p &= \sigma(W_1 x_{p-1} + W_2 s_{p-1} + b_1), \\ o_p &= f(W_3 x_{p-1} + W_4 s_{p-1} + b_2), \\ s_p &= (1 - u_p) \odot s_{p-1} + u_p \odot o_p, \end{aligned} \tag{6}$$

where $\odot$ is element-wise vector multiplication, σ is the sigmoid activation function, f is the exponential linear activation function, the W_* are weight matrices and the b_* are bias vectors. The output state s_p is both the output to the next layer and the input to state for the next recurrence.

D. Curriculum learning

We adopt a curriculum learning approach for the path finding, maze solving and circuit analysis tasks discussed in this work. To construct the curriculum, training examples are sorted into bins based on their size (number of nodes), with bin 1 being the smallest examples. We then define a probability distribution p_i over training example bins i. At each iteration of training, a minibatch is drawn from a bin with probability p_i. At the beginning of training, $p_1 = 1$ and $p_i = 0$ for all $i > 1$. Let ϕ be the index of the first bin with $p_\phi = 0$. After a fixed number of iterations, the bin probabilities are updated as:

$$p_i = p_i \eta + \frac{1 - \eta}{\phi}, \tag{7}$$

for all $i \leq \phi$, where η is a decay factor (0.25 in our experiments). All the p_i converge to $1/N$, where N is the number of bins. The entropy of this distribution increases monotonically, in accordance with the definition of curriculum learning [51].

E. Computing voltages in random circuits

To construct random circuit examples for the circuit analysis task, we started with a grid graph of $N \times N$ nodes. Each edge was subject to random deletion, with a variable probability of deletion depending on circuit size (from 0.1 for 2×2 grids to 0.5 for 8×8 and larger grids). For each remaining edge, a component was placed on that edge with the following probabilities: battery 0.05, resistor 0.7, wire 0.25. Batteries were placed with a random orientation of positive and negative terminals. If a battery was not present at the end of this procedure, a random edge was chosen and replaced with a battery. Properties for each component were also chosen randomly. For resistors, resistance was chosen from a uniform distribution between $100\,\Omega$ and $1000\,\Omega$. For batteries, voltage was chosen from a uniform distribution between $5\,\mathrm{V}$ and $20\,\mathrm{V}$.

All batteries were given an internal resistance of $100\,\Omega$ to ensure voltages and currents were well defined for all circuits. Operating point voltages were computed with PySpice [52], an interface to the commonly used Spice circuit simulation library [53]. Training examples were generated with grid sizes between 2×2 and 10×10. For each grid size, 500 training mini-batches were generated. Batch sizes decreased with sample size to accommodate the increased computation time required by the circuit analysis software. A single batch of 100 examples was generated for test data at each grid size from 2×2 to 15×15.

We converted PySpice circuits to graph examples for input to wave and graph convolution networks in four steps. First, (1) we labeled nodes with their voltages compared to ground (training targets) as determined by circuit analysis software PySpice. Second, (2) we labeled edges with two properties: resistance and input voltage. If an edge was a resistor it was labeled with 0 input voltage. If an edge was a wire, it was labeled with 0 input voltage and 0 resistance. Only battery edges had nonzero input voltage. Third, (3) we labeled nodes connected to the positive terminal of a battery to indicate battery orientation. A temperature encoding was used to indicate up to four batteries connected to the same node. Finally, (4) we converted the edge properties (input voltage and resistance) to a log space by the transformation $\log(1 + x)$.

IV. Results and Discussion

A. Finding paths in trees and graphs

Labeling nodes in a path between two other nodes requires propagating long-range information. This task is easily solved with a depth-first search; however, wave propagates information in a breath-first search, which does not start at either of the goal nodes. Nevertheless, we expected wave to outperform graph convolution in this task. To test this hypothesis, we constructed random path finding examples. For each example, a spanning tree was constructed from a grid graph via either a randomized depth first search (DFS) or a randomized Prim's algorithm [54]; edges in the spanning tree were retained as an undirected graph. Training data were randomly generated for grid graph sizes from 3×3 to 10×10 nodes. Each network architecture was provided the same features (the goal nodes), and the same output layer (dense layer with sigmoid activation). For this experiment, wave used a dense layer with $\tanh$ activation for its recurrent network. Wave used a node state size of 10. Graph convolution used a node state size of 5 and edge state size of 5. We trained both networks to label nodes traversed by the correct path using the cross entropy loss [55] with the Adam optimizer [56]. Optimizer parameters were: learning rate 1×10^{-3}, mini-batch size 50, and $30,000$ iterations. Evaluation was performed on a pre-generated set of 100 random trees for each grid graph size from 3×3 to 20×20. Each test network was evaluated on the same set of random trees.

Models were implemented in the Tensorflow deep learning framework [57]. Networks were trained with a curriculum learning scheme [51], starting with small graphs of size three (3×3 nodes) up to size 10 (Methods). Larger examples were introduced every $1,500$ iterations. Solutions were marked correct if the labeled path could be enumerated by starting at a goal node and transitioning to the adjacent node with maximum probability without backtracking (argmax solution).

A single wave forward-backward pass perfectly solved the DFS path finding task, both on trees of similar size as the training domain, and on trees much larger than those in the training data (Figure 2B). Furthermore, the solution learned by the wave network was generalized enough to achieve high performance on the Prim path finding task. In contrast, graph convolution with 5 or 10 passes was not able to solve the DFS task. Graph convolution with 10 passes performed worse on the Prim path finding task, suggesting that the solutions found by graph convolution may overfit the data. Examining the network outputs for an example from the Prim test, we see that while the wave network clearly marks path nodes, the graph convolutional network exhibits uncertainty in marking nodes that are far from a goal node (Figure 2A). In addition, the wave network had fewer parameters than the 10 pass graph convolution network (1641 vs. 2576). These findings support our hypothesis that wave propagates information more efficiently than graph convolution.

In addition to trees, a wave network with three forward-backward passes was able to find the shortest path in graphs with multiple possible paths between the start and end goal (Figure 2C). For this task, we generated mazes with multiple possible paths by starting with a DFS (or Prim) tree from the previous task, and connecting randomly chosen pairs of adjacent nodes in the grid. The resulting graphs were categorized by counting the number of resulting paths. Critically, wave was able to extrapolate from graphs with a few possible paths (one to four) to find shortest paths in graphs with many more possible paths (five to ten). Furthermore, wave was able generalize from mazes generated by the DFS algorithm to mazes generated with the Prim algorithm. In contrast, graph convolution with 5 passes was not able to achieve the same performance as wave on the training set, did not extrapolate well to the test data, and did not generalize well to mazes generated with the Prim algorithm

B. Solving a maze in an image

Solving mazes represented as images is much more difficult than path finding in a tree, because long-range information must be explicitly routed along valid paths, while avoiding walls. We translated the DFS and Prim graphs from the previous task into maze images. To create an undirected graph, the images were translated into pixel grid graphs, where adjacent pixels are connected by edges (Figure 3A). In these graphs, nodes are labeled as passable, wall or goal. Wave network messages propagate radially out from the center of the image, and are not restricted to the passable areas of the maze. The network must label the pixels that connect the two goals, without crossing a wall. Such a task is arguable more difficult for humans than image recognition; while we can tell

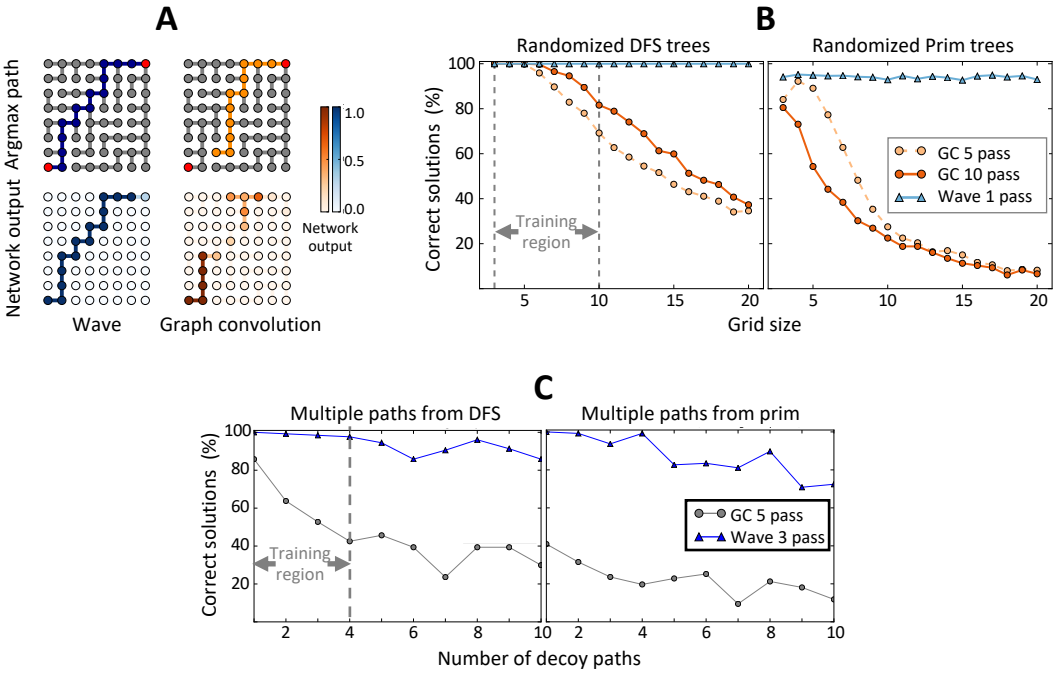

Fig. 2. **Wave networks label paths between distant nodes in trees and graphs.** (A) Graph convolution assigns low-confidence scores to nodes distant from the goals resulting in errors, while wave assigns high confidence to all path nodes. (B) A single wave forward-backward pass is sufficient to perfectly solve path-finding in trees constructed by randomized depth-first search, and generalizes to larger trees and to trees generated with a randomized Prim algorithm. In contrast, graph convolutional methods are unable to learn even the training data accurately. (C) In addition, multiple passes of wave can determine the shortest path in graphs with varying numbers of paths between the start and goal. Critically, wave models trained only on examples with fewer potential paths can extrapolate to graphs with far more potential paths.

at a glance that an image is a maze, solving a large maze requires meticulously tracing paths through the maze.

In addition to testing graph convolution and wave with fixed numbers of passes, we also introduced dynamic versions of these architectures, which perform $(N+1)/2$ passes for images of size $N \times N$. In the dynamic versions of the architectures, weights are shared between each pass, which substantially reduces the overall number of parameters. Finally, we also tested image convolutional neural networks with $3 \times 3 \times 16$ kernels, ELU activations, and max pooling at each pass. The network specifications and training procedures used for this task are identical to those from the previous section, except that models were trained for $60,000$ iterations and the curriculum learning scheme introduced larger mazes every $3,000$ iterations.

Wave with three or more passes substantially outperformed graph convolution and image convolution on both DFS and Prim mazes (Figure 3D). Examining network outputs showed that, while the dynamic wave was able to accurately label path nodes with high probability, graph convolution labeled many nodes outside of the correct path (Figure 3B). Using the dynamic wave, we generated outputs for varying numbers of passes. The algorithm learned by this network appears to progressively erase portions of the maze that are not part of the correct path, eventually converging when no non-path elements remain labeled (Figure 3C). We also note that the dynamic wave has the fewest parameters (1661) of all tested networks, with the exception of the dynamic graph convolution. These findings provide evidence that wave can propagate long-range information in images.

C. Computing voltages in a circuit

Computing voltages in a circuit is a more complex graph task than path finding in trees or images because it requires information propagation simultaneously between many interdependent nodes. The voltage at a node depends not only on its relationship with the ground node, but also on paths passing through batteries. We generated random circuits from grid graphs by randomly deleting edges, and replacing the remaining edges with one of three component types (wires, resistors or batteries) with randomly chosen properties (Figure 4A, Methods). Nodes connected to a battery's positive terminal were labeled to indicate battery orientation. One node was labeled ground. Voltages were calculated using PySpice [52]. Wave parameters were: state size 20, miniGRU recurrent unit. The miniGRU unit, a variant of the long short-term memory (LSTM) recurrent architecture (Methods), was used to improve long-range information propagation. In practice, we observed a small performance improvement on the circuit task using miniGRU compared to a simple recurrent neural network (data not shown). Graph convolution parameters were: node state size 15, edge state size 5. Training parameters were the same as previous tasks, except mini-batch size decreased with increasing example size to accommodate circuit simulator runtime (Methods). Models were trained for $60,000$ iterations

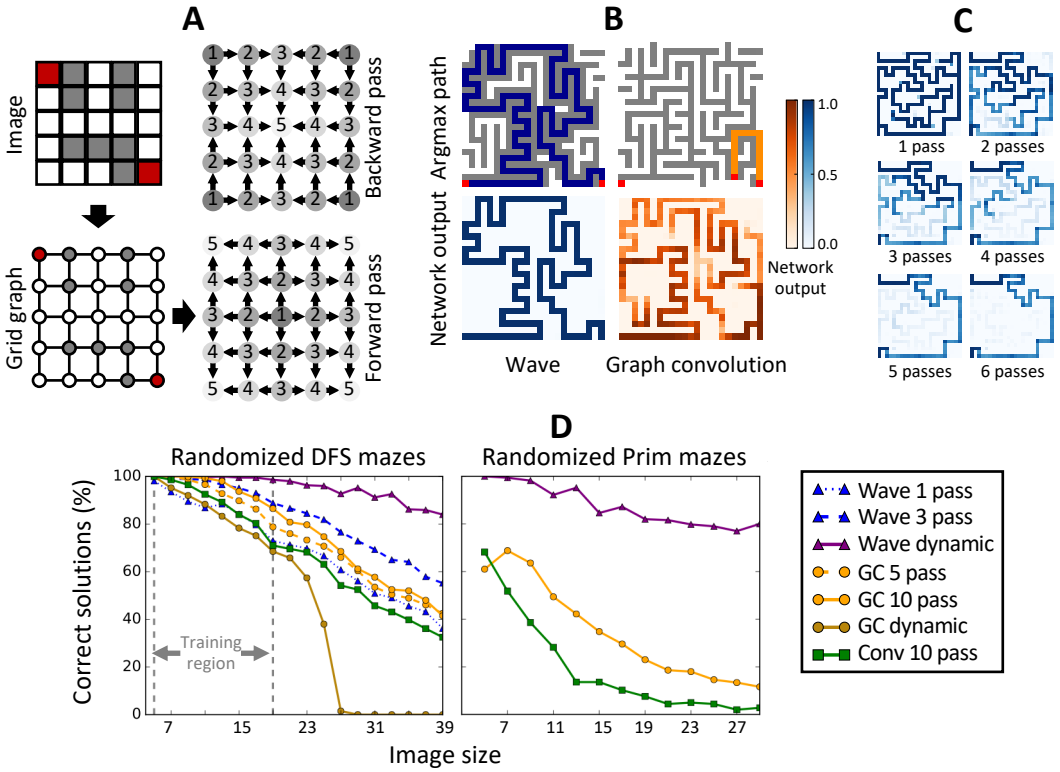

Fig. 3. **Wave networks solve mazes presented as images.** (A) We construct images of randomly generated mazes with labeled walls, start and end points, and converted these to grid graphs where each node represents a pixel and adjacent pixels are connected by edges. Wave processes the image in expanding and contracting shells. (B) Wave can find the correct path connecting the start and endpoints. In contrast, convolutional approaches label many decoy paths. (C) Dynamic wave learns a progressive erasing algorithm, where each successive pass removes weight from decoy paths. (D) Wave outperforms both graph convolution and image convolution with max pooling. A dynamic version of the wave, which adapts the number of passes to the image size, substantially outperforms all other tested methods and generalizes to randomized Prim mazes.

with larger circuit examples introduced every $4,000$ iterations. The training set included $4,500$ circuits ranging in size from 2×2 nodes to 10×10 nodes, while the test set included 100 random circuits of each size from 2×2 to 15×15 for a total of $1,400$ test circuits (Methods).

All tested wave networks achieved lower root-mean-square error compared to graph convolutional networks (Figure 4B). Wave with two or more passes exhibit a substantially slower growth in error rates with circuit size, even for circuits larger than those presented during training. Interestingly, increasing the number of convolutional passes did not improve the performance of graph convolution on this task. An individual circuit example reveals that errors made by graph convolution calculations are substantially biased for nodes far from the ground node, while wave errors are small, and unbiased (Figure 4A). In addition, when examining circuits of size 10, graph convolution errors rose in proportion with target node distance from the ground node, while wave errors exhibited substantially lower correlation with distance (Figure 4C). The 1 pass wave had substantially fewer parameters than the best graph convolution (12261 vs 20376), but still outperformed graph convolution on all tested examples sizes. These findings provide evidence that wave can propagate information among multiple, interdependent nodes over long ranges more efficiently than graph convolution.

V. Conclusion

Wave efficiently represents long-range dependencies in graphs for several different tasks when compared to graph convolution. Wave can (1) label paths connecting distant vertices in undirected graphs, (2) label paths in mazes represented as images, and (3) compute voltage potentials in circuits represented as graphs. Furthermore, wave can extrapolate from small training examples to much larger testing examples. In all of these experiments, the top wave networks also had the fewest parameters. While this study only examined relatively small graphs (up to 400 nodes), the computational and memory complexity of wave enable it to scale to much larger graphs. In particular, while graph convolution requires the number of passes to increase with graph radius to ensure information propagation between all nodes, wave can propagate information between all nodes in a graph with only a few passes. We are optimistic that wave may efficiently solve many tasks where representing long-range dependencies in graphs is critical.

Acknowledgments

The authors declare that they have no competing financial conflicts of interests. The authors are also grateful to the de-

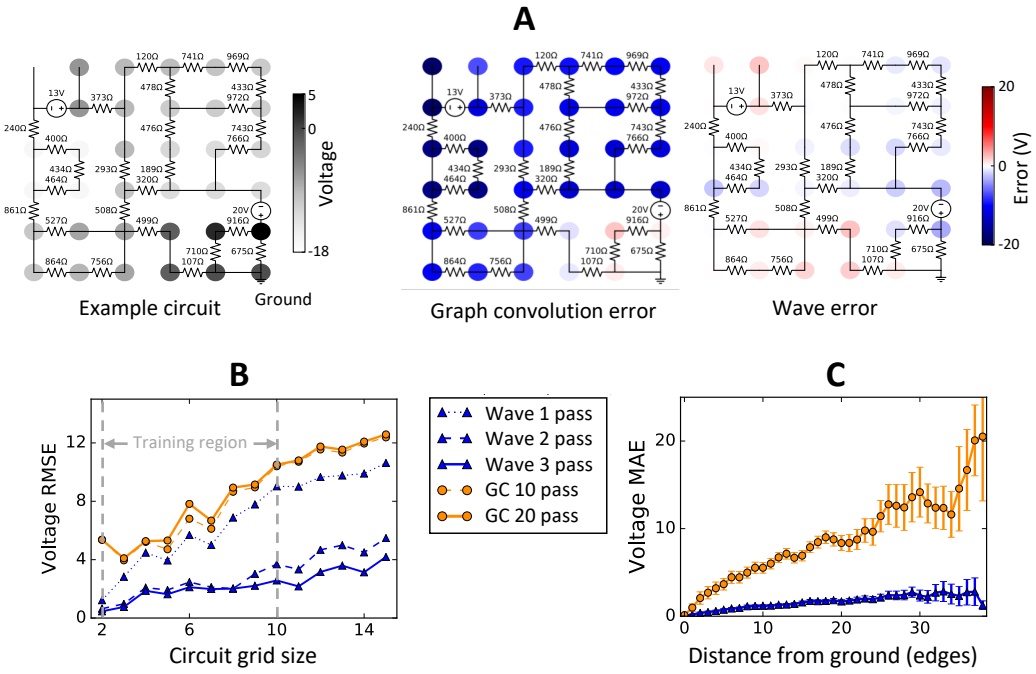

Fig. 4. **Wave networks compute voltages in a circuit.** (A) We constructed randomized circuits consisting of a single ground node (lower right), and randomly selected components (batteries, resistors and wires). The task was to label each junction node with its voltage compared to the ground node. Wave exhibited low error rates compared to graph convolutional approaches. (B) Graph convolution root-mean-square error (RMSE) increased quickly with circuit size compared to wave with more than one pass. Additional convolutional passes did not improve performance on this task. (C) Graph convolution exhibits increasing mean-absolute error (MAE) with distance from the ground node.

velopers of Tensorflow and the open-source cheminformatics tools OpenBabel and RDKit. Research reported in this publication was supported by the National Library Of Medicine of the National Institutes of Health under Award Numbers R01LM012222 and R01LM012482, and by the National Institutes of Health under Award Number GM07200. The content is the sole responsibility of the authors and does not necessarily represent the official views of the National Institutes of Health. Computations were performed using the facilities of the Washington University Center for High Performance Computing, which were partially funded by NIH grants nos. 1S10RR022984-01A1 and 1S10OD018091-01. We also thank both the Department of Immunology and Pathology at the Washington University School of Medicine, the Washington University Center for Biological Systems Engineering and the Washington University Medical Scientist Training Program for their generous support of this work.

References

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, may 2015. [Online]. Available: http://dx.doi.org/10.1038/nature14539

[2] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural Networks*, vol. 61, pp. 85–117, 2015.

[3] P. Baldi and G. Pollastri, "The Principled Design of Large-Scale Recursive Neural Network Architectures–DAG-RNNs and the Protein Structure Prediction Problem," *Journal of Machine Learning Research*, vol. 4, no. Sep, pp. 575–602, 2003.

[4] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to Forget: Continual Prediction with LSTM," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, oct 2000.

[5] D. K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams, "Convolutional networks on graphs for learning molecular fingerprints," in *Advances in neural information processing systems*, 2015, pp. 2224–2232.

[6] S. Kearnes, K. McCloskey, M. Berndl, V. Pande, and P. Riley, "Molecular graph convolutions: moving beyond fingerprints," *J. Comput.-Aided Mol. Des.*, vol. 30, no. 8, pp. 595–608, 2016.

[7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in neural information processing systems*, 2012, pp. 1097–1105.

[8] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *International Conference on Medical image computing and computer-assisted intervention*. Springer, 2015, pp. 234–241.

[9] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei, "Large-scale video classification with convolutional neural networks," in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, 2014, pp. 1725–1732.

[10] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-net: Fully convolutional neural networks for volumetric medical image segmentation," in *3D Vision (3DV), 2016 Fourth International Conference on*. IEEE, 2016, pp. 565–571.

[11] S. Pereira, A. Pinto, V. Alves, and C. A. Silva, "Brain tumor segmentation using convolutional neural networks in mri images," *IEEE transactions on medical imaging*, vol. 35, no. 5, pp. 1240–1251, 2016.

[12] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst, "Geometric deep learning: Going beyond euclidean data," *IEEE Signal Processing Magazine*, vol. 34, no. 4, pp. 18–42, jul 2017. [Online]. Available: https://doi.org/10.1109/msp.2017.2693418

[13] L. Page, S. Brin, R. Motwani, and T. Winograd, "The pagerank citation ranking: Bringing order to the web," 1998.

[14] N. Shibata, Y. Kajikawa, Y. Takeda, and K. Matsushima, "Detecting emerging research fronts based on topological measures in citation networks of scientific publications," *Technovation*, vol. 28, no. 11, pp. 758–775, 2008.

[15] Y. Ding, E. Yan, A. Frazho, and J. Caverlee, "Pagerank for ranking au-

thors in co-citation networks," *Journal of the Association for Information Science and Technology*, vol. 60, no. 11, pp. 2229–2243, 2009.
[16] J. Scott, *Social network analysis*. Sage, 2017.
[17] J.-D. J. Han, N. Bertin, T. Hao, D. S. Goldberg, G. F. Berriz, L. V. Zhang, D. Dupuy, A. J. Walhout, M. E. Cusick, F. P. Roth *et al.*, "Evidence for dynamically organized modularity in the yeast protein–protein interaction network," *Nature*, vol. 430, no. 6995, p. 88, 2004.
[18] D. Szklarczyk, A. Franceschini, S. Wyder, K. Forslund, D. Heller, J. Huerta-Cepas, M. Simonovic, A. Roth, A. Santos, K. P. Tsafou, M. Kuhn, P. Bork, L. J. Jensen, and C. von Mering, "STRING v10: protein–protein interaction networks, integrated over the tree of life," *Nucleic Acids Research*, vol. 43, no. D1, pp. D447–D452, oct 2014. [Online]. Available: https://doi.org/10.1093/nar/gku1003
[19] D. R. Zerbino and E. Birney, "Velvet: algorithms for de novo short read assembly using de bruijn graphs," *Genome research*, vol. 18, no. 5, pp. 821–829, 2008.
[20] N. M. O'Boyle, M. Banck, C. A. James, C. Morley, T. Vandermeersch, and G. R. Hutchison, "Open babel: An open chemical toolbox," *Journal of cheminformatics*, vol. 3, no. 1, p. 33, 2011.
[21] J. W. Raymond and P. Willett, "Maximum common subgraph isomorphism algorithms for the matching of chemical structures," *Journal of computer-aided molecular design*, vol. 16, no. 7, pp. 521–533, 2002.
[22] T. Lengauer, *Combinatorial algorithms for integrated circuit layout*. Springer Science & Business Media, 2012.
[23] T. L. Magnanti and R. T. Wong, "Network design and transportation planning: Models and algorithms," *Transportation science*, vol. 18, no. 1, pp. 1–55, 1984.
[24] M. K. Matlock, N. L. Dang, and S. J. Swamidass, "Learning a local-variable model of aromatic and conjugated systems," *ACS Central Science*, vol. 4, no. 1, pp. 52–62, jan 2018. [Online]. Available: https://doi.org/10.1021/acscentsci.7b00405
[25] B. Gallagher and T. Eliassi-Rad, "Leveraging label-independent features for classification in sparsely labeled networks: An empirical study," in *Advances in Social Network Mining and Analysis*. Springer, 2010, pp. 1–19.
[26] K. Henderson, B. Gallagher, L. Li, L. Akoglu, T. Eliassi-Rad, H. Tong, and C. Faloutsos, "It's who you know: graph mining using recursive structural features," in *Proceedings of the 17th ACM SIGKDD international conference on knowledge discovery and data mining*. ACM, 2011, pp. 663–671.
[27] J. Zaretzki, M. Matlock, and S. J. Swamidass, "Xenosite: accurately predicting cyp-mediated sites of metabolism with neural networks," *Journal of chemical information and modeling*, vol. 53, no. 12, pp. 3373–3383, 2013.
[28] M. K. Matlock, T. B. Hughes, and S. J. Swamidass, "XenoSite server: A web-available site of metabolism prediction tool," *Bioinformatics*, vol. 31, no. 7, pp. 1136–1137, 2015.
[29] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk," in *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining - KDD '14*. ACM Press, 2014. [Online]. Available: https://doi.org/10.1145/2623330.2623732
[30] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
[31] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "Line: Large-scale information network embedding," in *Proceedings of the 24th international conference on world wide web*. International World Wide Web Conferences Steering Committee, 2015, pp. 1067–1077.
[32] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '16*. ACM Press, 2016. [Online]. Available: https://doi.org/10.1145/2939672.2939754
[33] T. Hofmann, B. Schölkopf, and A. J. Smola, "Kernel methods in machine learning," *The annals of statistics*, pp. 1171–1220, 2008.
[34] S. V. N. Vishwanathan, N. N. Schraudolph, R. Kondor, and K. M. Borgwardt, "Graph kernels," *Journal of Machine Learning Research*, vol. 11, no. Apr, pp. 1201–1242, 2010.
[35] M. Niepert, M. Ahmed, and K. Kutzkov, "Learning convolutional neural networks for graphs," in *International conference on machine learning*, 2016, pp. 2014–2023.
[36] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," *arXiv preprint arXiv:1511.05493*, 2015.
[37] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
[38] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," *arXiv preprint arXiv:1704.01212*, 2017.
[39] K. S. Tai, R. Socher, and C. D. Manning, "Improved semantic representations from tree-structured long short-term memory networks," *arXiv preprint arXiv:1503.00075*, 2015.
[40] P. Baldi, "The inner and outer approaches to the design of recursive neural architectures," *Data mining and knowledge discovery*, vol. 32, no. 1, pp. 218–230, 2018.
[41] A. Graves, G. Wayne, and I. Danihelka, "Neural turing machines," *arXiv preprint arXiv:1410.5401*, 2014.
[42] A. Graves, G. Wayne, M. Reynolds, T. Harley, I. Danihelka, A. Grabska-Barwińska, S. G. Colmenarejo, E. Grefenstette, T. Ramalho, J. Agapiou *et al.*, "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, no. 7626, p. 471, 2016.
[43] Y. Ma, Z. Guo, Z. Ren, Y. E. Zhao, J. Tang, and D. Yin, "Dynamic graph neural networks," *ArXiV*, vol. abs/1810.10627, 2018. [Online]. Available: http://arxiv.org/abs/1810.10627
[44] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. F. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner, Ç. Gülçehre, F. Song, A. J. Ballard, J. Gilmer, G. E. Dahl, A. Vaswani, K. Allen, C. Nash, V. Langston, C. Dyer, N. Heess, D. Wierstra, P. Kohli, M. Botvinick, O. Vinyals, Y. Li, and R. Pascanu, "Relational inductive biases, deep learning, and graph networks," *ArXiV*, vol. abs/1806.01261, 2018. [Online]. Available: http://arxiv.org/abs/1806.01261
[45] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel, "Neural relational inference for interacting systems," *arXiv preprint arXiv:1802.04687*, 2018.
[46] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
[47] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (elus)," *arXiv preprint arXiv:1511.07289*, 2015.
[48] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, 2010, pp. 249–256.
[49] R. Jozefowicz, W. Zaremba, and I. Sutskever, "An empirical exploration of recurrent network architectures," in *International Conference on Machine Learning*, 2015, pp. 2342–2350.
[50] K. Cho, B. van Merrienboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder–decoder approaches," in *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*. Association for Computational Linguistics, 2014, pp. 103–111. [Online]. Available: http://www.aclweb.org/anthology/W14-4012
[51] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th annual international conference on machine learning*. ACM, 2009, pp. 41–48.
[52] F. Salvaire, "Pyspice," https://pyspice.fabrice-salvaire.fr, 2018.
[53] T. L. Quarles, "Analysis of performance and convergence issues for circuit simulation," California University Berkeley Electronics Research Lab, Tech. Rep., 1989.
[54] J. Kleinberg and E. Tardos, *Algorithm design*. Pearson Education India, 2006.
[55] J. Shore and R. Johnson, "Properties of cross-entropy minimization," *IEEE Trans. Inf. Theory*, vol. 27, no. 4, pp. 472–482, 1981.
[56] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," vol. arXiv:1412.6980, 2014.
[57] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, and M. Isard, "Tensorflow: A system for large-scale machine learning," in *Proceedings of the 12th USENIX conference on operating systems design and implementation*, vol. 16, 2016, pp. 265–283.